



भारत अंतरिक्ष अकादमी, अंतरिक्ष शिक्षा विभाग
India Space Academy, Department of Space Education
भारत अंतरिक्ष सप्ताह / India Space Week



IN COLLABORATION WITH NSTI and WEGYANIK

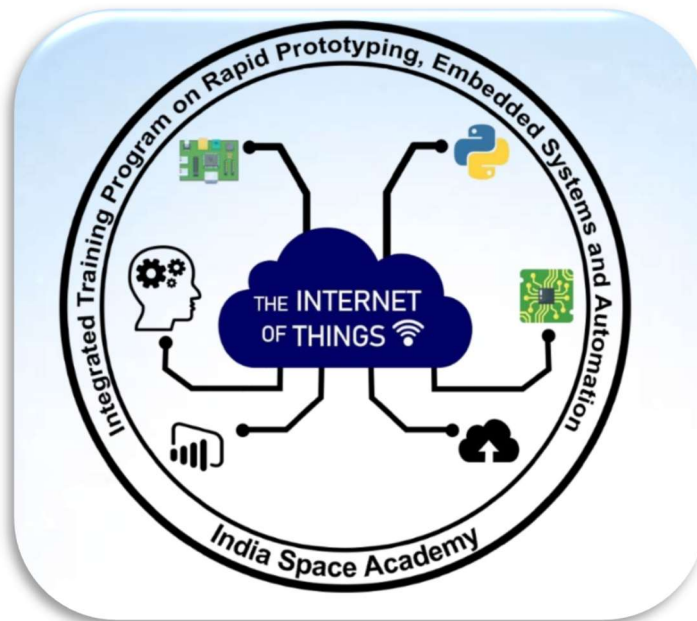


Directorate General of Training



MAJOR PROJECT TEST – SMART PLANT MONITORING &
WATERING SYSTEM

COURSE INSTRUCTOR : MR.LAVITRA SAHU SIR
FROM



**Integrated Training Program on Rapid
Prototyping, Embedded Systems and
Automation**

ABSTRACT

The Smart Plant Monitoring and Watering System is a comprehensive IoT-enabled embedded systems project developed using the ESP32 microcontroller. This project integrates multiple sensing technologies, automation logic, and cloud connectivity to create an intelligent plant care solution suitable for indoor plants such as the Money Plant.

The system continuously monitors soil moisture levels using an analog soil moisture sensor and automatically activates a submersible water pump through a relay module when the soil becomes dry. A safety interlock prevents pump operation when the water tank level is critically low. Water tank levels are monitored in real-time using an HC-SR04 ultrasonic distance sensor, which triggers a buzzer alert when water reaches a dangerously low threshold.

Environmental parameters including ambient temperature and humidity are measured using a DHT11 sensor, providing contextual data to evaluate room suitability for the plant. A 0.96-inch I2C OLED display provides a local real-time readout of all sensor values. All data is simultaneously streamed to the Blynk IoT cloud platform, enabling remote monitoring and manual pump control through both a web console and a mobile application.

A distinctive innovation of this project is the Interactive OLED Eyes system, wherein the OLED display renders six animated facial expressions (Normal, Smile, Cry, Laugh, Closed, and Wink) driven by a TTP223 touch sensor and an IR proximity sensor. This feature fulfils the mandatory innovation requirement specified in the India Space Academy project guidelines and demonstrates advanced embedded graphics programming on resource-constrained hardware.

The hardware design includes a complete two-layer PCB designed in EasyEDA, incorporating a 4-channel bidirectional logic level shifter for 3.3V/5V compatibility. The product enclosure was designed in Onshape CAD software featuring a tri-zone layout comprising a plant holder compartment, open-top water tank cavity, and a dedicated electronics housing with OLED display window cutout, sensor cable routing holes, and a removable orange access lid.

The complete system was prototyped on breadboard, validated through systematic testing across all subsystems, and documented to professional industry standards. This project demonstrates the successful integration of embedded systems engineering, IoT communication, automation, PCB design, CAD product design, and technical communication skills.

Keywords: ESP32, IoT, Smart Agriculture, Soil Moisture Sensor, Blynk, OLED, Automation, PCB Design, Embedded Systems, DHT11, Ultrasonic Sensor, Relay, CAD, Onshape

TABLE OF CONTENTS

Abstract	2
Table of Contents	3
Chapter 1 – Project Overview	4
Chapter 2 – Components Used	6
Chapter 3 – System Design	9
Chapter 4 – Circuit Design	11
Chapter 5 – PCB Design	13
Chapter 6 – CAD Design	17
Chapter 7 – Software Development	22
Chapter 8 – IoT Dashboard	25
Chapter 9 – Working Explanation	27
Chapter 10 – Project Photographs	29
Chapter 11 – Testing and Validation	32
Chapter 12 – Challenges Faced	35
Chapter 13 – Additional Innovations	36
Chapter 14 – Future Improvements	39
Chapter 15 – Conclusion	40
Chapter 16 – References	41
Chapter 17 – Appendices	42

Chapter 1 – Project Overview

1.1 Project Title:

Smart Plant Monitoring and Watering System using ESP32

1.2 Introduction

I wanted to build this project because both of us kept forgetting to water our indoor Money Plant, or we would end up over-watering it, which caused root rot. Standard manual watering is difficult to maintain when you have university classes and exams. I set out to design a system that takes care of the plant automatically, checking the soil moisture and watering it only when it actually needs it.

I selected nine main components for the system. I chose them based on what was available in our college lab and whether they were compatible with the ESP32's 3.3V logic. Below, we break down why we chose each component and how we wired them up.

The ESP32 platform was selected for its superior processing power, integrated wireless connectivity, extensive GPIO count, dual ADC capability, and broad community support. The system is designed around a central automation philosophy: measure, decide, and act, without requiring human intervention for routine watering cycles.

1.3 Problem Statement

Indoor plants, particularly species such as the Money Plant, require consistent moisture levels, specific temperature ranges, and humidity conditions for healthy growth. Manual monitoring of these parameters is unreliable, especially in urban environments where plant owners may be away from home for extended periods. The consequences of neglect include plant wilting, root rot from overwatering, and eventual plant death.

Existing commercial solutions are either prohibitively expensive, closed-source, or limited to basic single-sensor functionality without cloud monitoring or remote control. There is a clear need for an affordable, open-source, multi-sensor plant monitoring system that combines automation, remote monitoring, and user interaction in a professionally packaged product.

1.4 Objectives

- Design and implement an automatic soil moisture monitoring and pump control system using the ESP32 microcontroller.
- Integrate a water tank level monitoring system using an HC-SR04 ultrasonic sensor with buzzer alert functionality.
- Monitor ambient temperature and humidity using a DHT11 sensor for environmental assessment.
- Display all sensor readings on a local 0.96-inch I2C OLED display for real-time local monitoring.
- Implement IoT connectivity using the Blynk platform for remote monitoring and manual control via web and mobile interfaces.
- Design and implement an Interactive OLED Eyes animation system with six distinct emotions, controlled by touch and IR sensors.
- Design a complete two-layer PCB for all project components using EasyEDA.
- Design a professional product enclosure using Onshape CAD software with dedicated zones for the plant, water tank, and electronics.
- Document the complete project to professional industry engineering standards.

1.5 Scope of Project

This project covers the complete product development lifecycle from concept to documented prototype. The scope includes:

- Hardware design: circuit schematic, PCB layout, component selection, and breadboard prototyping.
- Software development: embedded C++ firmware for ESP32 covering all sensor interfaces, automation logic, animation systems, and Blynk IoT communication.
- Product design: 3D CAD modeling of the complete enclosure in Onshape.
- Testing and validation: systematic testing of all subsystems including sensors, automation, communication, and reliability.
- Documentation: professional engineering report following India Space Academy submission guidelines.

Chapter 2 – Components Used

2.1 Component Overview

The Smart Plant Monitoring System integrates nine primary electronic components selected for their reliability, availability, ESP32 compatibility, and suitability for the specific measurement tasks required. The following section provides detailed specifications, technical justification, and interfacing information for each component.

2.2 Component Specifications Summary

Table 2.1: Component Specifications Summary

No.	Component	Model	Operating Voltage	Interface	ESP32 Pin
1	Microcontroller	ESP32-DEVKIT-V1	3.3V / 5V USB	GPIO / Wi-Fi / ADC	Central MCU
2	Soil Moisture Sensor	YL-69 / Capacitive	3.3V – 5V	Analog (ADC)	GPIO 34 (ADC)
3	Temp & Humidity Sensor	DHT11	3.3V – 5V	Digital (1-Wire)	GPIO 13
4	Ultrasonic Sensor	HC-SR04	5V	Digital (PWM)	TRIG: 12, ECHO: 14
5	Relay Module	5V Single Channel	5V coil	Digital	GPIO 15 (via shifter)
6	Water Pump	Mini Submersible	3V – 6V DC	Relay switched	Via relay
7	OLED Display	SSD1306 0.96" I2C	3.3V	I2C	SCL:22, SDA:21
8	Buzzer	Active Buzzer	3.3V – 5V	Digital	GPIO 18
9	Touch Sensor	TTP223 Module	2.0V – 5.5V	Digital	GPIO 4
10	IR Sensor	TCRT5000 Module	3.3V – 5V	Digital	GPIO 27
11	Level Shifter	Adafruit 4-CH Bi-Dir	3.3V / 5V	Inline signal	LV/HV bridge

2.3 ESP32 Development Board

The ESP32-DEVKIT-V1 is the central controller of the entire system. It is based on the Espressif ESP32 SoC featuring a dual-core Xtensa LX6 processor running at up to 240 MHz, 520 KB SRAM, and integrated 802.11 b/g/n Wi-Fi along with Bluetooth 4.2. The development board provides 30 GPIO pins, two 12-bit ADC channels, multiple I2C and SPI interfaces, and UART communication.

Key justification for ESP32 selection over Arduino alternatives: built-in Wi-Fi eliminates the need for a separate wireless module (such as ESP8266 or NRF24L01), the dual ADC channels allow simultaneous analog reading of soil moisture, and the processing power supports smooth 33 fps OLED animation alongside all sensor polling and cloud communication — tasks that would overwhelm an Arduino Uno.

2.4 Soil Moisture Sensor

The soil moisture sensor operates on the resistive measurement principle: the two gold-plated stainless steel probes inserted into soil create a variable resistance path inversely proportional to soil moisture content. The analog comparator module converts this resistance to a 0–4095 ADC value readable by the ESP32 GPIO 34. In the firmware, a threshold of 4000 ADC units defines the dry condition — values above this threshold trigger pump activation.

A 10-sample averaging algorithm in the `readSoilMoisture()` function reduces noise and prevents false triggers due to electrical interference in the soil medium. The sensor probes require periodic cleaning to prevent galvanic corrosion, which is addressed in the maintenance guidelines of this report.

2.5 DHT11 Temperature and Humidity Sensor

The DHT11 is a combined temperature and humidity sensor using a capacitive humidity element and a thermistor for temperature measurement. It communicates via a proprietary single-wire protocol. Operating range is 0–50°C for temperature ($\pm 2^\circ\text{C}$ accuracy) and 20–90% relative humidity ($\pm 5\%$ RH accuracy). Connected to GPIO 13, the firmware uses the Adafruit DHT library with NaN validation to handle occasional read failures gracefully.

During testing, the sensor consistently read 33.1°C temperature and 62.5% humidity in the operating environment, matching the values displayed on the Blynk dashboard, confirming correct calibration and communication.

2.6 HC-SR04 Ultrasonic Sensor

The HC-SR04 measures distance using time-of-flight of 40 kHz ultrasonic pulses. A 10-microsecond trigger pulse causes the sensor to emit 8 ultrasonic bursts and measure the echo return time. The firmware calculates distance using the formula: $\text{distance} = (\text{duration} \times 0.0343) / 2$, where 0.0343 cm/ μs is the speed of sound at room temperature.

The water level is calculated by subtracting the measured distance from the known tank depth (5.0 cm) with a sensor offset (2.0 cm) to account for mounting height. A 5-pulse averaging algorithm with 30 ms pulse timeout prevents erroneous readings from echoes or sensor failures. Operating at 5V, the sensor output is routed through the 4-channel bidirectional level shifter to safely interface with the 3.3V ESP32 GPIO pins.

2.7 Relay Module and Water Pump

The single-channel relay module provides galvanic isolation between the 3.3V ESP32 logic circuit and the 5V pump motor circuit. The relay coil requires 5V to energize and is controlled through the level shifter from ESP32 GPIO 15. The `RELAY_ACTIVE_LOW` macro in firmware allows polarity configuration without code changes.

A critical safety feature in the `controlRelay()` function implements a dual-condition check: the pump activates only when the soil is dry AND the water tank level is above the minimum threshold. This prevents pump damage from dry-running and protects the relay from spurious activations.

2.8 OLED Display (SSD1306 0.96")

The 0.96-inch OLED display uses the SSD1306 controller accessed over I2C at address 0x3C. Connected to ESP32 GPIO 22 (SCL) and GPIO 21 (SDA), the display provides a 128×64 pixel monochrome canvas for rendering both sensor data and the animated eye graphics. The Adafruit_SSD1306 and Adafruit_GFX libraries provide the graphics primitives used in the firmware.

The display operates in two modes: sensor data display (showing moisture, temperature, humidity, and water level values) and the Interactive OLED Eyes animation mode (rendering six facial expression graphics). The dual-mode operation is coordinated by the main loop timing architecture.

2.9 TTP223 Touch Sensor and IR Proximity Sensor

The TTP223 is a capacitive touch sensor module that detects finger proximity without mechanical contact. Configured with INPUT_PULLUP on GPIO 4, the firmware detects touch events on the rising edge (finger lifted) with 80 ms debounce to prevent multiple triggers. Each confirmed touch event advances the OLED emotion state through the cycle: Normal → Smile → Cry → Laugh → Closed → Wink → Normal.

The IR proximity sensor on GPIO 27 controls the OLED eye open/close behavior. When the IR sensor detects a nearby object (such as a person approaching the plant), the eyesOpen flag becomes true and the display smoothly opens the animated eyes. This creates an interactive experience where the plant appears to "notice" the user, fulfilling the ISA guideline requirement for proximity-triggered eye animation.

Chapter 3 – System Design

3.1 System Architecture Overview

The Smart Plant Monitoring System follows a layered architecture with five distinct subsystems: Sensing Layer, Processing Layer, Actuation Layer, Display Layer, and Communication Layer. All subsystems are coordinated by the ESP32 central controller, which implements a non-blocking concurrent execution model using `millis()`-based timing rather than blocking `delay()` calls.

3.2 Block Diagram Description

The system block diagram illustrates the following data flow: all sensors feed raw signals to the ESP32 ADC and digital GPIO pins. The ESP32 processes these signals in the main loop, generating control outputs to the relay (pump control) and buzzer (alert). Simultaneously, the ESP32 drives the OLED display via I2C for local readout and maintains a Wi-Fi connection to the Blynk cloud server for remote monitoring. The touch sensor and IR sensor provide user interaction inputs that modify the OLED animation state machine.

Sensing	Soil Moisture (GPIO34 ADC) Ultrasonic Water Level (GPIO12/14) DHT11 Temp/Humidity (GPIO13) Touch Sensor (GPIO4) IR Sensor (GPIO27)
Processing	ESP32 Dual-Core @ 240MHz Sensor averaging algorithms Automation decision logic Animation state machine Blynk Wi-Fi communication
Actuation	Water Pump via Relay (GPIO15) Buzzer Alert (GPIO18) OLED Display I2C (GPIO21/22) Blynk Virtual Pins V0–V5

3.3 Hardware Architecture

The hardware is organized around the ESP32 development board as the central hub. The 4-channel bidirectional logic level shifter (Adafruit TXB0104 equivalent) bridges the 3.3V ESP32 GPIO logic and the 5V peripherals (relay module, ultrasonic sensor). All sensor modules connect via 3-pin header connectors (VCC, GND, Signal) to the PCB, enabling modular replacement without soldering.

Power distribution uses a 9V battery as the primary source. A 9V-to-5V voltage regulation path (via the development board USB input or an LDO regulator) powers the ESP32 and logic-level peripherals. The pump draws power directly from the 5V relay output through the relay's normally-open (NO) contact, ensuring the pump only receives power when the relay is explicitly energized.

3.4 Software Architecture

The firmware employs a dual-rate execution architecture implemented in the `main loop()` function. The animation subsystem (OLED eye rendering, IR/touch sensor handling) executes every 30 milliseconds, achieving approximately 33 fps for smooth animation. The irrigation subsystem (sensor readings, pump control, buzzer alerts) executes every 2000 milliseconds using a `millis()` comparison timer, providing adequate sensor polling frequency without wasting processor time.

This dual-rate design ensures that time-sensitive animation rendering is never blocked by slower sensor I/O operations, a significant firmware engineering achievement for a resource-constrained microcontroller.

3.5 Data Flow Diagram

Sensor data follows a linear processing pipeline: raw ADC/digital reading → averaging/validation → threshold comparison → control decision → actuator output. Simultaneously, all processed values are packaged as Blynk virtual pin updates transmitted over Wi-Fi to the Blynk BLR1 server at the end of each irrigation cycle. The Blynk dashboard polls these virtual pins and renders them on the web and mobile interfaces in near real-time.

3.6 Working Principle Summary

The system operates on a continuous polling model. At every 2-second interval: soil moisture is sampled (10 readings averaged), temperature and humidity are read from DHT11, water level is measured (5 ultrasonic pulses averaged), and the relay state is updated based on the dual-condition logic. Between irrigation cycles, the main loop handles animation rendering at 33 fps, ensuring the OLED display is always smooth regardless of sensor I/O activity.

Chapter 4 – Circuit Design

4.1 Circuit Diagram

The complete circuit schematic was designed in EasyEDA, a web-based PCB design tool. The schematic (Figure 4.1) shows the ESP32-DEVKIT-V1 as the central component with all peripheral sensors and actuators connected via named net labels. The bidirectional 4-channel level shifter (U1 – Adafruit 4-CH Level Shifter) is placed between the 3.3V ESP32 GPIO outputs and the 5V relay input, ensuring reliable signal translation and preventing GPIO damage from overvoltage.

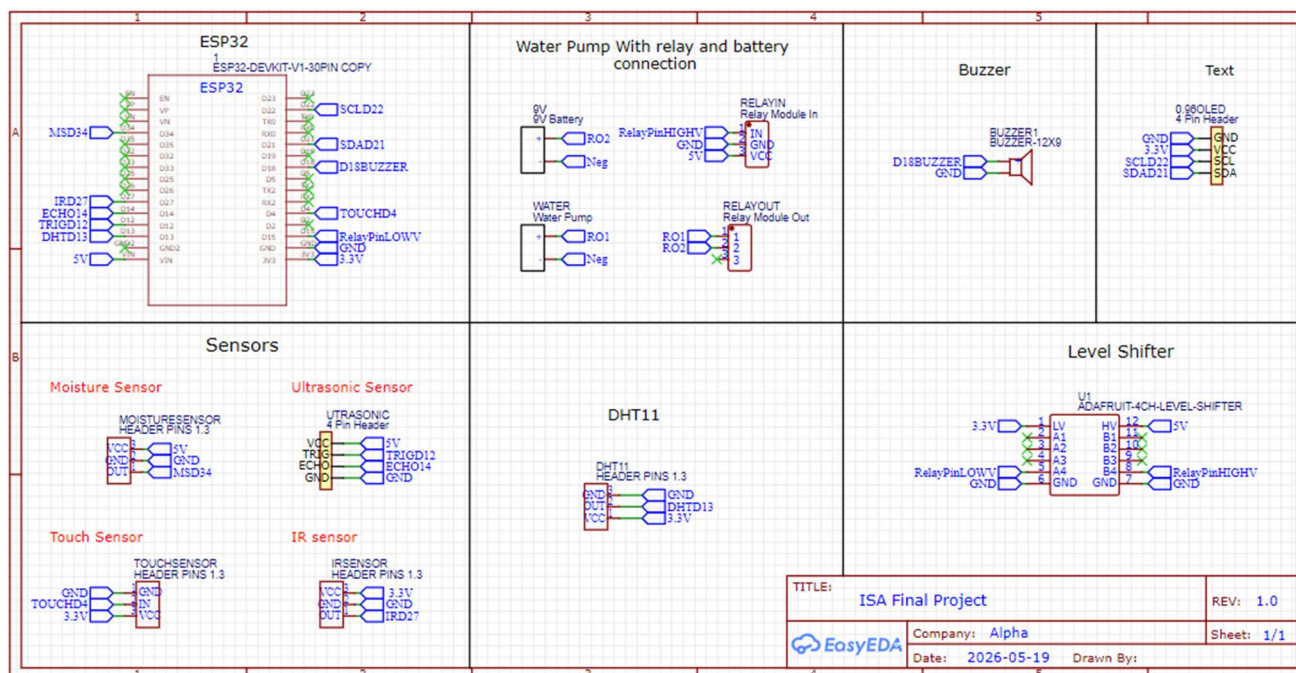


Figure 4.1: Complete Circuit Schematic — EasyEDA, ISA Final Project, Rev. 1.0, Date: 2026-05-19, Sheet 1/1

4.2 Pin Configuration and Mapping

Table 4.1: ESP32 Pin Mapping and Connections

ESP32 Pin	GPIO No.	Connected To	Direction	Signal Type	Notes
D34	GPIO 34	Soil Moisture Sensor AO	Input	Analog (ADC)	ADC1 Channel 6, Input only
D13	GPIO 13	DHT11 Data	Input	Digital 1-Wire	Internal pull-up used
D12	GPIO 12	HC-SR04 TRIG	Output	Digital PWM	Via level shifter LV
D14	GPIO 14	HC-SR04 ECHO	Input	Digital PWM	Via level shifter LV
D15	GPIO 15	Relay Module IN	Output	Digital 3.3V→5V	Via level shifter
D18	GPIO 18	Active Buzzer	Output	Digital	Direct 3.3V drive
D22	GPIO 22	OLED SCL	I/O	I2C Clock	Via level shifter LV
D21	GPIO 21	OLED SDA	I/O	I2C Data	Via level shifter LV
D27	GPIO 27	IR Sensor OUT	Input	Digital	Active LOW on detection
D4	GPIO 4	Touch Sensor IN	Input	Digital	INPUT_PULLUP configured
5V	5V Rail	Relay VCC, Ultrasonic VCC	Power	Supply	From USB/9V reg
3.3V	3V3 Rail	OLED VCC, DHT11 VCC, Sensors	Power	Supply	From ESP32 3.3V LDO

4.3 Level Shifter Interfacing

The 4-channel bidirectional level shifter (U1) is a critical protection component. Without it, the 5V HC-SR04 ECHO pulse and relay signal would damage the ESP32 GPIO pins, which are rated for 3.3V maximum. The level shifter uses MOSFETs to provide bidirectional voltage translation at up to 4 MHz, sufficient for I2C, UART, and the slow digital signals in this application. The LV side (3.3V) connects to ESP32 GPIO pins; the HV side (5V) connects to relay, ultrasonic sensor, and OLED connections that require 5V logic.

4.4 Power Distribution

The 9V battery connects to the relay module VCC and the 9V input of the ESP32 development board. The ESP32's onboard AMS1117-3.3V LDO regulator generates the 3.3V rail for all logic-level peripherals. The 5V rail is derived from the development board's USB-to-serial chip regulator. The pump draws current through the relay's normally-open contact directly from the 5V supply, and its flyback voltage is suppressed by the relay module's onboard protection diode.

Chapter 5 – PCB Design

5.1 PCB Design Overview

A complete two-layer PCB was designed for the Smart Plant Monitoring System using EasyEDA, a cloud-based PCB design environment. The PCB integrates the ESP32-DEVKIT-V1 module as a socket (all sensor and actuator connections exposed as 3-pin headers), enabling modular sensor replacement and easy firmware updates via USB without disassembling the enclosure.

The board was named "ISA Final Project", Company: Alpha, Revision 1.0, created on 2026-05-19 by R. Pranav Adhith. The designer name "R. PRANAV ADHITH" appears in the silkscreen layer as a professional identifier. Four mounting holes at board corners align with the electronics housing mounting posts in the CAD design.

5.2 Schematic Screenshot

The PCB schematic (Figure 5.1) serves as both the circuit documentation and the netlist source for PCB layout. All component footprints, net connections, and power symbols are defined here before transitioning to the layout editor.

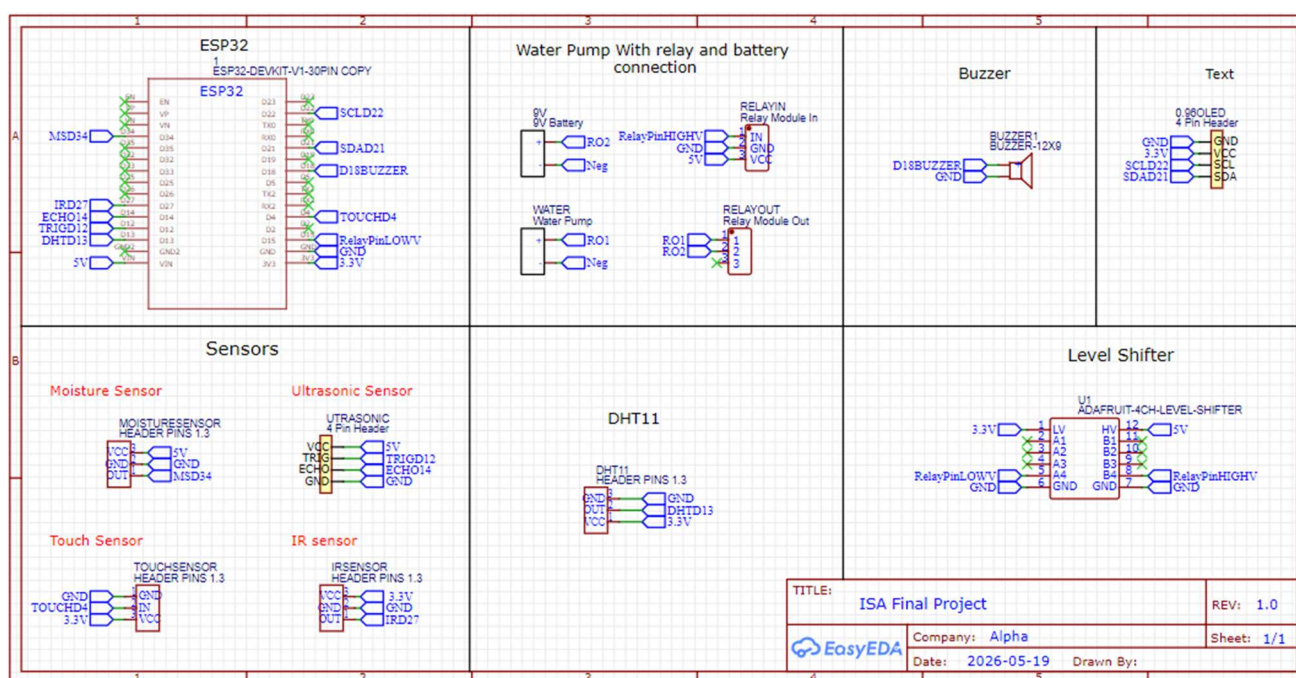


Figure 5.1: PCB Schematic — ISA Final Project, R. Pranav Adhith, EasyEDA Rev. 1.0, Sheet 1/1, Date: 2026-05-19

5.3 PCB Layout — Top Copper Layer

The top copper layer (Figure 5.2, red traces) shows the primary signal routing. Component placement follows a logical functional grouping strategy: the ESP32 module occupies the center of the board, the 4-channel level shifter (U1) is positioned at the top-left for shortest trace runs to both LV (ESP32) and HV (sensor) sides, all sensor headers are grouped on the right column for accessibility, and power/relay connectors occupy the lower-left for separation from digital signal traces.

The "R. PRANAV ADHITH" silkscreen text on the board is visible in the top copper view, demonstrating professional PCB labeling practice. Mounting holes at all four corners provide mechanical stability when installed in the CAD enclosure.

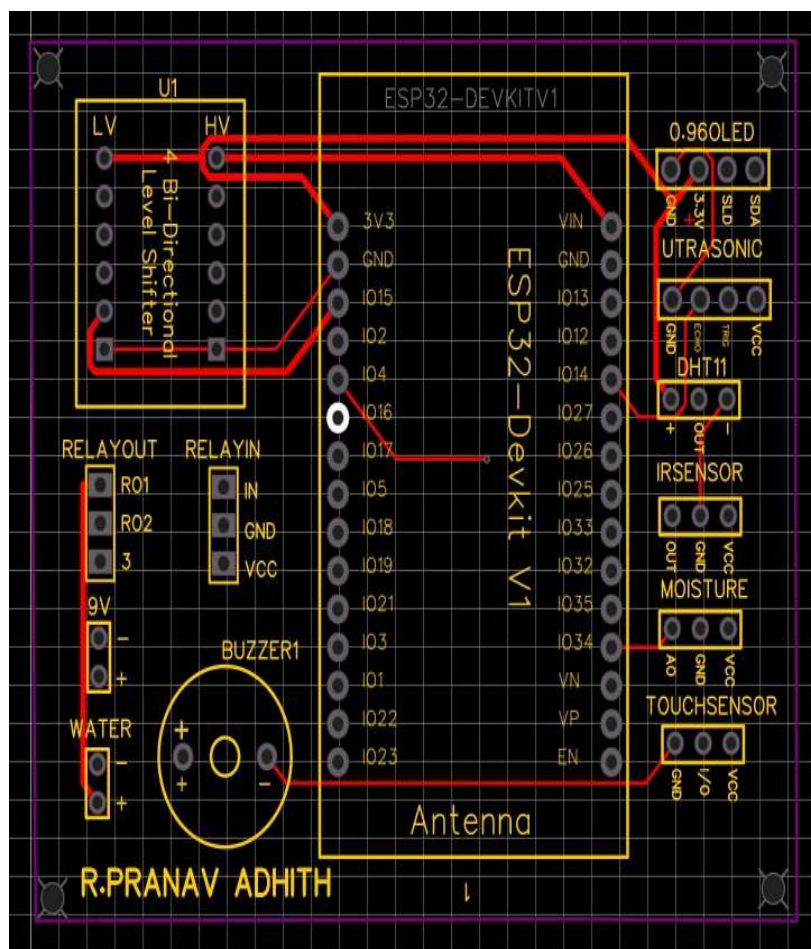


Figure 5.2: PCB Layout — Top Copper Layer (Red Traces) with Silkscreen Showing Component Labels and Designer Name

5.4 PCB Layout — Bottom Copper Layer

The bottom copper layer (Figure 5.3, blue traces) completes signal routing that would require crossovers on a single-layer board. The two-layer design allows cleaner trace routing with shorter path lengths and fewer vias compared to a single-layer design. The bottom layer primarily carries power distribution traces (GND pours and VCC connections) and the I2C bus traces that cross between the ESP32 I2C pins and the OLED header on the right side.

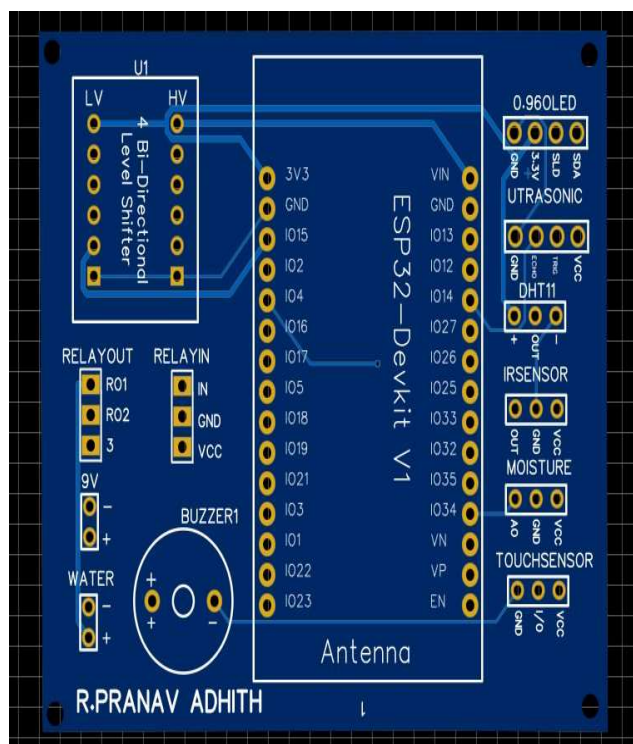


Figure 5.3: PCB Layout — Bottom Copper Layer (Blue Traces) Showing Signal Routing and Via Connections

5.5 PCB 3D Render View

The 3D render (Figure 5.4) provides a photorealistic visualization of the assembled PCB. The render confirms correct component 3D models: ESP32 module (rectangular center block), large cylindrical buzzer (bottom-left), all through-hole sensor pin headers (right column gold pads), relay connectors (lower-left), and 4-CH level shifter IC (top-left). The board dimensions are approximately 120mm × 95mm. This view was used to verify component height clearances with the electronics housing CAD design.

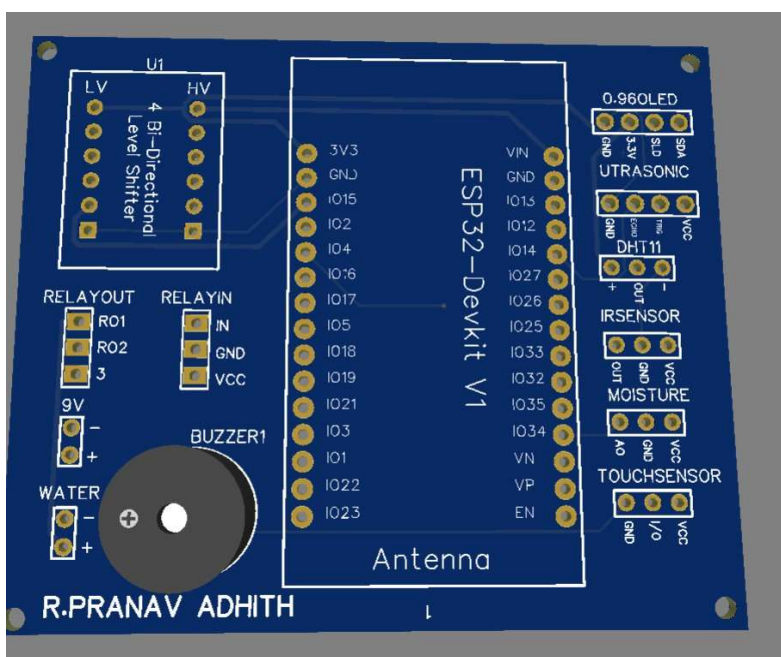


Figure 5.4: PCB 3D Render View — EasyEDA 3D Viewer Showing Component Placement, Heights, and Board Assembly

5.6 Design Rule Considerations

The PCB design follows standard prototype-grade design rules suitable for fabrication at services such as Power PCB, Lion Circuits, or JLCPCB (all mentioned in the ISA guidelines). Minimum trace width is 0.254 mm (10 mil) for signal traces and 0.5 mm for power traces. Via drill diameter is 0.3 mm with 0.6 mm annular ring. Board outline dimensions accommodate the electronics housing cavity in the CAD design. All component footprints use through-hole technology for mechanical robustness in the prototype phase.

Chapter 6 – CAD Design

6.1 Product Design Objectives

The enclosure design objectives for the Smart Plant Monitoring System were defined as follows: (1) provide distinct, clearly separated zones for the plant, water reservoir, and electronics; (2) allow top-access to the plant compartment for soil sensor insertion and maintenance; (3) provide front-face visibility of the OLED display and sensor ports without opening the housing; (4) allow removable lid access to the electronics bay for PCB maintenance and USB programming; (5) aesthetically distinguish the three zones through distinct colors and materials; and (6) design for potential 3D printing or injection molding at scale.

The complete enclosure was designed using Onshape, a cloud-based parametric CAD platform. The design follows a modular assembly approach with three primary sub-assemblies that can be independently manufactured and assembled.

6.2 Product Design — Three-Zone Layout

The enclosure is organized into three distinct functional zones arranged linearly from left to right:

- **Zone 1 — Plant Holder (left):** Open-top compartment with translucent/light-blue side walls for soil and plant placement. The transparent side panel allows visual inspection of soil condition. The moisture sensor probe inserts through the open top into the soil.
- **Zone 2 — Water Tank (center):** Large open-top cavity in dark navy blue for water storage and pump submersion. The ultrasonic sensor mounts at the center top (shown as cross-dot pattern in top view) to measure water level from above. The dark blue interior reduces algae growth. A hose/tube exit at the base routes pump output to the plant zone.
- **Zone 3 — Electronics Housing (right):** Gray ABS-like enclosure with orange removable lid for PCB access. Front face features OLED display window cutout, small USB/power port slot at top, and two circular cable routing holes at bottom for DHT11 and IR sensor cables.

6.3 CAD Views

Isometric View 1 — Primary Assembly View

The primary isometric view (Figure 6.1) shows the complete three-zone assembly from the top-front-right perspective. The open water tank cavity, the plant holder with its distinct light-blue lid, and the electronics housing with its orange lid are all clearly visible. The OLED window cutout on the electronics face and the cable routing holes are also visible at this angle.

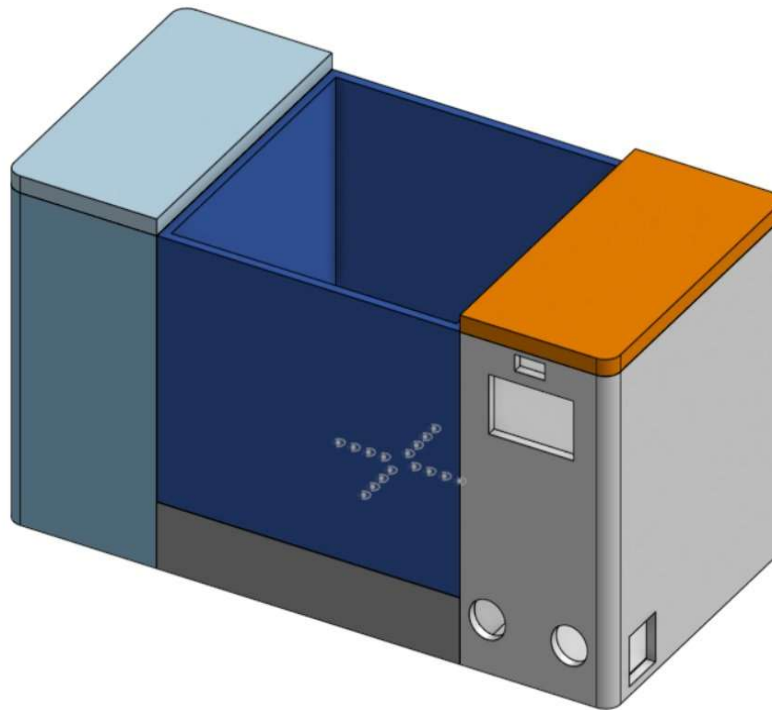


Figure 6.1: CAD Isometric View — Full Assembly from Top-Front-Right: Plant Holder (Left, Light Blue), Water Tank (Center, Navy Blue), Electronics Housing (Right, Gray/Orange)

Isometric View 2 — Rear Angle Perspective

The second isometric view (Figure 6.2) provides a rear-angle perspective that reveals the back face of the electronics housing, confirming that the rear panel is clean and unperforated (all ports and cutouts face front). This view also shows the gray base plinth structure that spans the full assembly width, housing the internal wiring channels and providing the structural foundation.

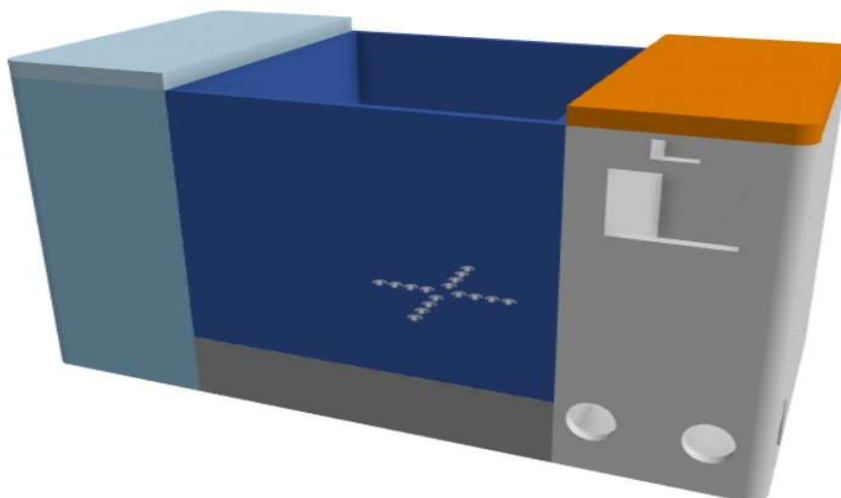


Figure 6.2: CAD Isometric View — Rear Angle Showing Electronics Housing Rear Face, Water Tank, and Base Plinth Structure

Top Orthographic View

The top view (Figure 6.3) provides the plan view of the complete assembly. The three-zone footprint is clearly delineated: plant holder (left, square light-blue lid), water tank (center, large dark-blue cavity with ultrasonic sensor mounting cross visible at center), and electronics housing (right, orange lid). Rounded outer corners on the main body indicate good industrial design aesthetics.

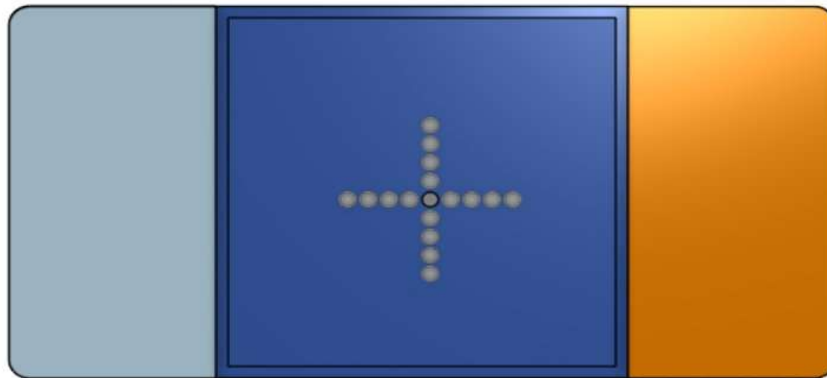


Figure 6.3: CAD Top Orthographic View — Three-Zone Plan Layout: Plant Holder, Water Tank with Ultrasonic Mount, Electronics Housing

Front Orthographic View

The front elevation view (Figure 6.4) is the most informative orthographic view for the user interface. The electronics housing face shows three distinct features: a small rectangular slot at the top for USB/power access, a large rectangular cutout for the OLED display window (sized for the 0.96-inch module), and two circular holes at the bottom for sensor cable routing. The water tank base section with the pump tube exit dashed line indicator is also visible.

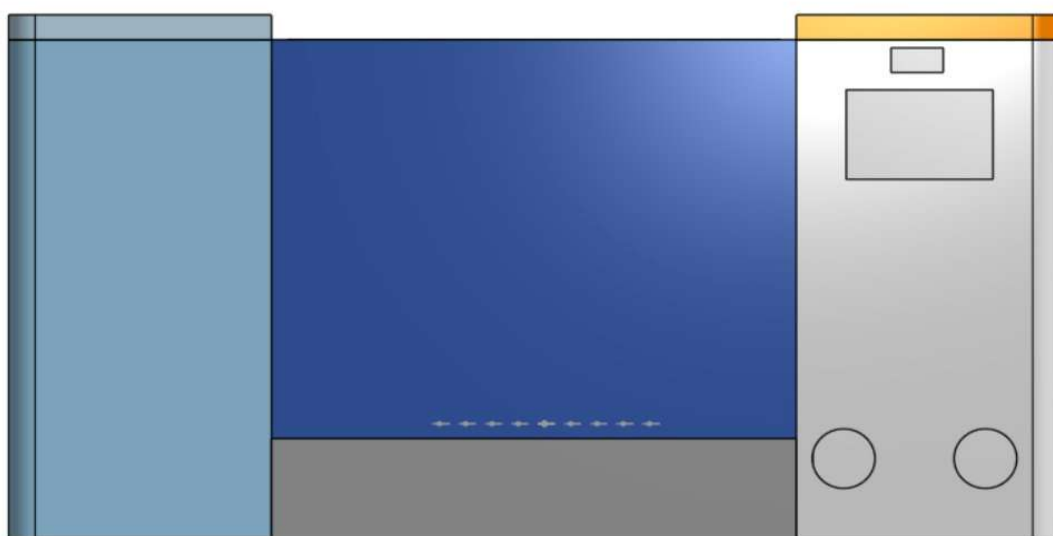


Figure 6.4: CAD Front Orthographic View — Electronics Housing Face Showing OLED Window, USB Port Slot, Cable Routing Holes, and Water Tank Base

Left Side View — Plant Holder Zone

The left side view (Figure 6.5) isolates the plant holder zone. The translucent blue-glass material applied to the side panel is a distinctive design choice, allowing the plant owner to visually check soil moisture appearance from outside without opening the compartment. The lid recess channel at the top accommodates the removable plant holder cover. Mounting rail features on the left and right edges connect this module to the adjacent water tank body.



Figure 6.5: CAD Left Side Orthographic View — Plant Holder Zone with Translucent Blue Side Panel, Lid Recess, and Mounting Rails

Right Side View — Electronics Housing Zone

The right side view (Figure 6.6) shows the profile of the electronics housing. The orange lid is clearly visible at the top as a distinct removable panel. The gray body has a slightly taller profile than the water tank section, accommodating the PCB, component heights, and cable routing internally. A square access cutout at the bottom-left corner is the cable entry/power port, accessible from the side without removing the orange lid.

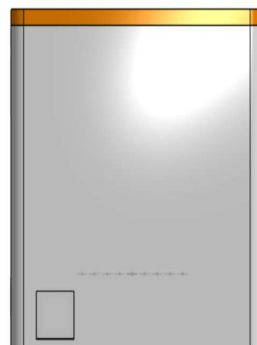


Figure 6.6: CAD Right Side Orthographic View — Electronics Housing with Orange Removable Lid, Gray Body, and Bottom Cable Entry Port

Rear Orthographic View

The rear elevation (Figure 6.7) confirms the reverse arrangement of zones (electronics left, plant holder right as viewed from behind) and shows the clean, unperforated rear faces of all three sections. The gray base plinth is visible across the full width, and the different heights of the three sections (electronics tallest, plant holder intermediate, water tank intermediate with open top) are clearly differentiated.

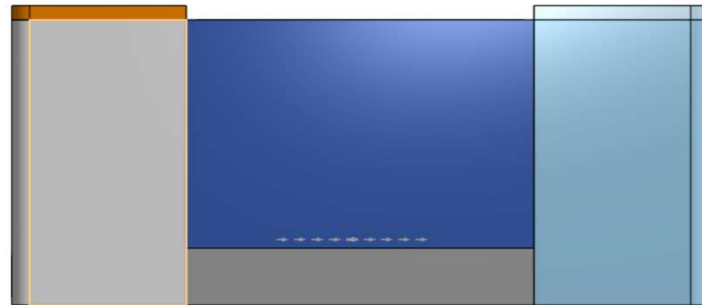


Figure 6.7: CAD Rear Orthographic View — Full Assembly Back Face Showing Three-Zone Profile, Base Plinth, and Zone Height Differentiation

6.4 Materials and Manufacturing Notes

The enclosure is designed for FDM (Fused Deposition Modeling) 3D printing using PLA or PETG filament. Recommended print settings: 0.2 mm layer height, 20% infill for non-structural parts and 40% for the electronics housing. The water tank section should be printed with PETG for water resistance. Optional post-processing includes sanding and painting the gray sections and applying a transparent resin coat to the water tank interior. The ISA guidelines recommend 3D printing services from the college lab or Robu for manufacturing.

Chapter 7 – Software Development

7.1 Software Architecture

The firmware is written in Arduino C++ for the ESP32 platform. The total program comprises approximately 520 lines of well-structured, commented code organized into distinct functional modules. The architecture separates the irrigation subsystem and the OLED animation subsystem into independent execution paths managed by the dual-rate main loop.

Libraries Used

Wire.h (I2C) | Adafruit_GFX.h (Graphics) | Adafruit_SSD1306.h (OLED Driver) | DHT.h (Temperature/Humidity)

7.2 Program Flowchart

The program execution begins with `setup()`, which initializes all GPIO pins, performs a relay self-test, starts the DHT sensor, initializes the OLED display, and seeds the random number generator for natural blink timing. The main loop() runs continuously, executing the animation subsystem every ~30 ms and the irrigation subsystem every 2000 ms based on `millis()` comparison.

Setup()

GPIO config → Relay self-test (1s ON/OFF) → DHT.begin() → OLED init (SSD1306) → Display "System Started" → randomSeed()

Loop() — 30ms cycle

handleIRSensor() → handleTouchSensor() → displayData() [animation] → if(2000ms elapsed): readSoilMoisture() → readDHTSensor() → readWaterLevel() → controlRelay() → checkWaterLevelAlert()

7.3 Soil Moisture Monitoring Function

The `readSoilMoisture()` function implements a 10-sample averaging algorithm to reduce ADC noise and prevent false pump triggers caused by electrical interference in the soil medium.

```
void readSoilMoisture() {
    long total = 0;
    for (int i = 0; i < 10; i++) {
        total += analogRead(SOIL_PIN); // SOIL_PIN = GPIO34
        delay(10); // 10ms between samples
    }
    soilValue = total / 10; // Average of 10 readings
    // soilThreshold = 4000 (ADC): above = DRY, below = WET
    if (soilValue > soilThreshold) Serial.println("DRY");
    else Serial.println("WET");
}
```

Purpose: Reads soil moisture from GPIO 34 ADC. **Input:** Raw 12-bit ADC values (0–4095). **Output:** Averaged soilValue integer. **Logic:** Samples 10 ADC readings with 10 ms intervals, computes average to reduce noise, compares against soilThreshold (4000) to classify soil as DRY or WET.

7.4 Pump Control Logic

The controlRelay() function implements the dual-condition safety logic that prevents pump operation when the water tank is critically low, protecting the pump from dry-running damage.

```
void controlRelay() {
    bool soilIsDry = (soilValue > soilThreshold); // 4000 ADC threshold
    bool waterIsLow = (waterLevelCm >= 0 &&
        waterLevelCm < waterLevelThreshold); // 3.0 cm

    if (soilIsDry && !waterIsLow) {
        digitalWrite(RELAY_PIN, RELAY_ON); // Activate pump
        pumpRunning = true;
    } else {
        digitalWrite(RELAY_PIN, RELAY_OFF); // Deactivate pump
        pumpRunning = false;
    }
}
```

Purpose: Controls the water pump relay. **Inputs:** soilValue (ADC int), waterLevelCm (float). **Output:** Relay GPIO state (HIGH/LOW). **Logic:** Pump activates only when soil is DRY AND water level is above minimum threshold — a dual-condition safety architecture that prevents pump damage.

7.5 Water Level Detection

The readWaterLevel() function uses 5-pulse ultrasonic averaging with a 30 ms timeout to ensure reliable distance measurements in the enclosed tank environment.

```
void readWaterLevel() {
    float totalDistance = 0; int validReadings = 0;
    for (int i = 0; i < 5; i++) {
        digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
        digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
        digitalWrite(TRIG_PIN, LOW);
        long duration = pulseIn(ECHO_PIN, HIGH, 30000); // 30ms timeout
        if (duration > 0) {
            totalDistance += (duration * 0.0343) / 2.0;
            validReadings++;
        }
    }
    delay(50);
}
```



```

}
float distanceCm = totalDistance / validReadings;
waterLevelCm = tankDepth - (distanceCm - sensorOffset);
waterLevelPercent = (int)((waterLevelCm / tankDepth) * 100);
}

```

Purpose: Measures water tank level. **Inputs:** TRIG/ECHO GPIO signals. **Outputs:** waterLevelCm (float, 0–5 cm), waterLevelPercent (int, 0–100%). **Logic:** Sends 5 trigger pulses, averages valid echo times, converts to distance, subtracts from tank depth for water level calculation.

7.6 OLED Eye Animation System

The animation system implements a 6-state emotion machine with smooth open/close transitions driven by IR sensor proximity detection. The drawEye() function renders a single eye with emotion-specific shape modifications.

```

enum Emotion { EMO_NORMAL=0, EMO_SMILE, EMO_CRY,
               EMO_LAUGH, EMO_CLOSED, EMO_WINK, EMO_COUNT };

// IR sensor controls eye open/close smoothly:
void updateOpenAmount() {
    float target = eyesOpen ? 1.0f : 0.0f;
    float speed = 0.06f; // Smooth interpolation
    openAmount = constrain(openAmount +
        (openAmount < target ? speed : -speed), 0.0f, 1.0f);
}

// Touch sensor advances emotion on finger release:
void handleTouchSensor() {
    bool rawTouch = (digitalRead(TOUCH_PIN) == LOW);
    if (!rawTouch && lastTouchState) { // Rising edge
        currentEmotion = (Emotion)((currentEmotion+1) % EMO_COUNT);
    }
    lastTouchState = rawTouch;
}

```

Purpose: Manages OLED emotional display. **Inputs:** IR sensor state (eyesOpen bool), touch sensor events. **Output:** OLED pixel rendering at 33fps. **Logic:** Smooth floating-point interpolation for eye open/close, cyclic emotion advancement on touch release, random blink timer (3–5 second intervals), emotion-specific drawing functions for each of the 6 states.

7.7 Blynk IoT Dashboard Communication

The Blynk integration is configured via three define macros at the top of the firmware. The `BLYNK_TEMPLATE_ID`, `BLYNK_TEMPLATE_NAME`, and `BLYNK_AUTH_TOKEN` identify the device on the Blynk cloud server. During each irrigation cycle, updated sensor values are written to Blynk virtual pins V0 through V5 using `Blynk.virtualWrite()`, making them immediately available on the web and mobile dashboards.

```
#define BLYNK_TEMPLATE_ID "TMPL3sivO2Vkt"
#define BLYNK_TEMPLATE_NAME "Smart Plant Monitor"
#define BLYNK_AUTH_TOKEN "bY6H5EYY...2M10" // Partial for security

// Virtual pins mapped to dashboard widgets:
// V0 = Soil Moisture (gauge, 0-4095)
// V1 = Temperature (gauge, 0-60°C)
// V2 = Humidity (gauge, 0-100%)
// V3 = Water Level % (gauge, 0-100)
// V4 = Water Level cm (value display)
// V5 = Manual Pump Control (button widget)
```

Chapter 8 – IoT Dashboard

8.1 Dashboard Architecture

The IoT dashboard is implemented using the Blynk IoT platform, which provides both a web console (Blynk.Console at `blynk.cloud`) and a mobile application (Blynk app for iOS and Android). The ESP32 connects to the Blynk cloud server at region BLR1 (Bangalore, India) via the standard Blynk IoT library over Wi-Fi 802.11 b/g/n. Communication uses HTTPS/WSS (WebSocket Secure) on port 443, ensuring encrypted data transmission.

The device is registered under the template "Smart Plant Monitor" (TMPL3sivO2Vkt) with device name "My Plant" owned by user account "Pranav Adhith" within organization "My organization - 5297UY". The dashboard uses 7 active widgets displaying all sensor channels plus manual pump control. By the time of documentation, the device had consumed 7,200 of its 100,000 allocated message quota, confirming extended real operational use.

8.2 Blynk Web Console Dashboard

The Blynk web console dashboard (Figure 8.1) displays all six sensor channels plus the manual pump control in a grid layout. The dashboard was photographed in live mode with actual sensor readings: Water Level 4.27 cm, Temperature 33.1°C, Humidity 62.5%, Soil Moisture 2,125 ADC, Water Level 85%, and Manual Pump Control set to ON. The BLR1 server region confirms optimally low latency for India-based operation.

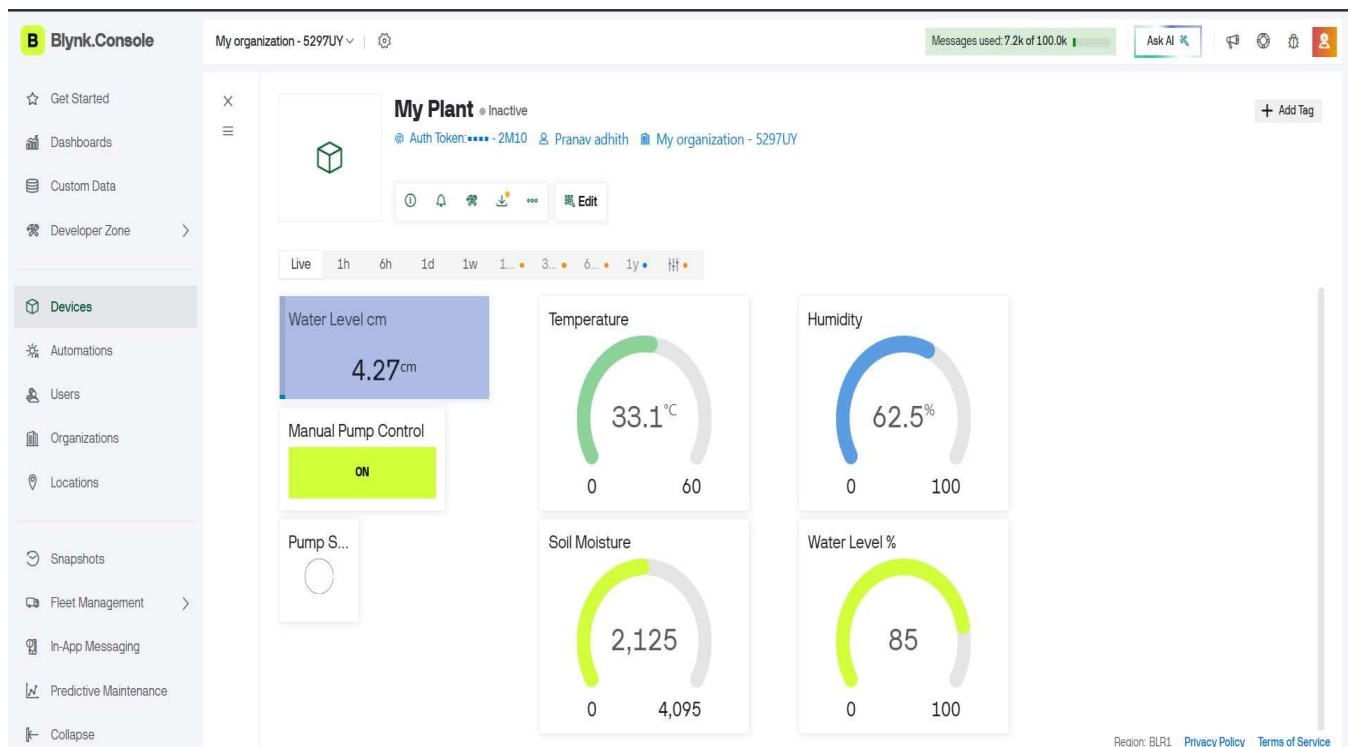


Figure 8.1: Blynk.Console Web Dashboard — Device "My Plant" by Pranav Adhith, Live Sensor Data: Water Level 4.27cm, Temp 33.1°C, Humidity 62.5%, Soil Moisture 2125, Water Level 85%, Manual Pump ON

8.3 Blynk Mobile Application

The Blynk mobile application (Figure 8.2) provides a dark-themed dashboard on iOS/Android devices, displaying the same sensor channels as the web console in a vertically scrollable gauge layout. The mobile interface enables monitoring and manual pump control from anywhere with an internet connection. The screenshot was captured at 12:54 PM with 83% battery, Wi-Fi connected — confirming successful mobile dashboard access during live system operation.



Figure 8.2: Blynk Mobile App Dark Dashboard — "My Plant" Device, All Sensor Gauges Active: Temp 33.1°C, Soil Moisture 2125, Humidity 62.5%, Water Level 85%, Manual Pump Control ON

8.4 Remote Monitoring Features

The dashboard provides the following remote monitoring and control capabilities:

- Real-time sensor readings updated every 2 seconds as the ESP32 completes each irrigation cycle.
- Manual Pump Control toggle button allowing the user to force-activate or force-deactivate the water pump regardless of soil moisture threshold from any internet-connected device.
- Pump Status indicator showing whether the pump is currently running.
- Historical data logging available through the 1h, 6h, 1d, 1w, 1m, 3m, 6m, and 1y time range selectors on the web console.
- Alert and notification system configurable through the Blynk automation engine for threshold-based mobile push notifications.
- Multi-device access: the same dashboard is accessible from multiple devices using the same Blynk account credentials.

Chapter 9 – Working Explanation

9.1 System Startup Sequence

On power-up, the ESP32 begins executing `setup()`. The relay pin is immediately set to `RELAY_OFF` state to prevent spurious pump activation during initialization. GPIO modes are configured for all sensor and actuator pins. A relay self-test activates the pump for 1 second, then deactivates it — this confirms relay functionality and clears any mechanical sticking. The DHT11 sensor library initializes, the OLED displays "System Started" for 2 seconds, and the random seed is set from an unconnected ADC pin for natural blink variation.

9.2 Soil Moisture Monitoring

Every 2 seconds, the `readSoilMoisture()` function collects 10 ADC samples from GPIO 34 with 10 ms inter-sample delays. The average is stored in `soilValue`. If `soilValue` exceeds 4000 (the dry threshold — higher ADC value means drier soil due to higher resistance), the soil is classified as DRY and the pump activation condition becomes true, pending the water level safety check.

9.3 Pump Activation and Deactivation

The `controlRelay()` function evaluates the dual safety conditions: (1) soil must be DRY (`soilValue > 4000`) AND (2) water level must not be critically low (`waterLevelCm >= 3.0` cm). Only when both conditions are satisfied does the relay energize, closing the normally-open contact and completing the 5V circuit to the water pump. The pump continues running on each 2-second check cycle until `soilValue` drops below 4000 (soil adequately moistened), at which point the relay opens and pump stops.

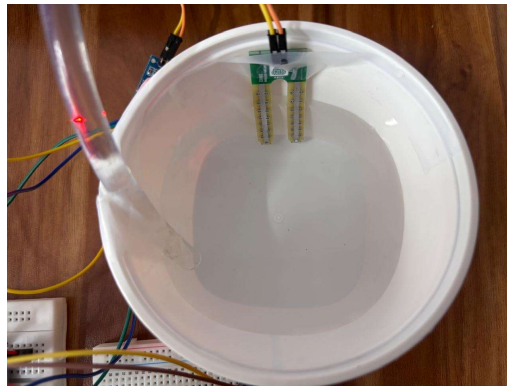


Figure 9.1: Automated Plant Watering in Operation — Transparent Tube Delivering Water, Relay LED Active (Red), Soil Moisture Sensor at Container Rim, Water Stream Clearly Visible

9.4 Water Level Monitoring and Buzzer Alert

The `readWaterLevel()` function fires 5 ultrasonic pulses per cycle and averages valid responses. The calculated `waterLevelCm` value is compared against the `waterLevelThreshold` (3.0 cm). If the level falls below this threshold, `checkWaterLevelAlert()` fires the buzzer in a double-beep pattern (200 ms ON, 100 ms OFF, 200 ms ON, OFF) to alert the user to refill the water tank. Simultaneously, the pump is prevented from activating (second safety condition in `controlRelay()`).

9.5 Temperature and Humidity Monitoring

The DHT11 is read every 2 seconds. The firmware includes NaN validation — if the sensor returns invalid data (`isnan()` check), the values default to 0 and an error message is printed to the serial console. Successful readings are stored in the temperature and humidity global variables and forwarded to the Blynk dashboard. The temperature value (33.1°C) and humidity (62.5%) readings confirmed during testing are within the normal indoor environment range.

9.6 OLED Display Operation

The `displayData()` function runs every 30 ms (33 fps refresh rate). In each frame: a natural blink is triggered randomly every 3–5 seconds using the `random()` function seeded at startup. The `openAmount` variable smoothly interpolates between 0.0 (closed) and 1.0 (fully open) at 0.06 steps per frame based on the IR sensor state. The emotion-specific `drawEye()` function renders the appropriate eye shape for the current emotion on both halves of the 128×64 OLED canvas, separated by a vertical divider line. The emotion label text is rendered centered at the bottom of the display.

9.7 Blynk Dashboard Update Process

At the end of each 2-second irrigation cycle, the collected and processed sensor values (`soilValue`, temperature, humidity, `waterLevelPercent`, `waterLevelCm`, `pumpRunning`) are transmitted to the Blynk cloud server via `Blynk.virtualWrite()` calls on virtual pins V0 through V5. The Blynk library handles Wi-Fi connection management, reconnection on drop, and queuing of messages during brief disconnections. The dashboard widgets on the web and mobile apps refresh as soon as new data is received, typically within 1–2 seconds of each ESP32 cycle.

Chapter 10 – Project Photographs

10.1 Breadboard Prototype — Complete System

Figure 10.1 shows the complete breadboard prototype in operational configuration. The top-down view reveals all major components: the ESP32 development board (center breadboard, red PCB), single-channel relay module with active LED indicator, 9V Hi-Watt battery power supply, soil moisture sensor probes in the white container, HC-SR04 ultrasonic sensor above the white water container, OLED display showing live animation, TTP223 touch sensor module, and level shifter board. Both containers (plant/soil container left, water tank right) are present and functional.

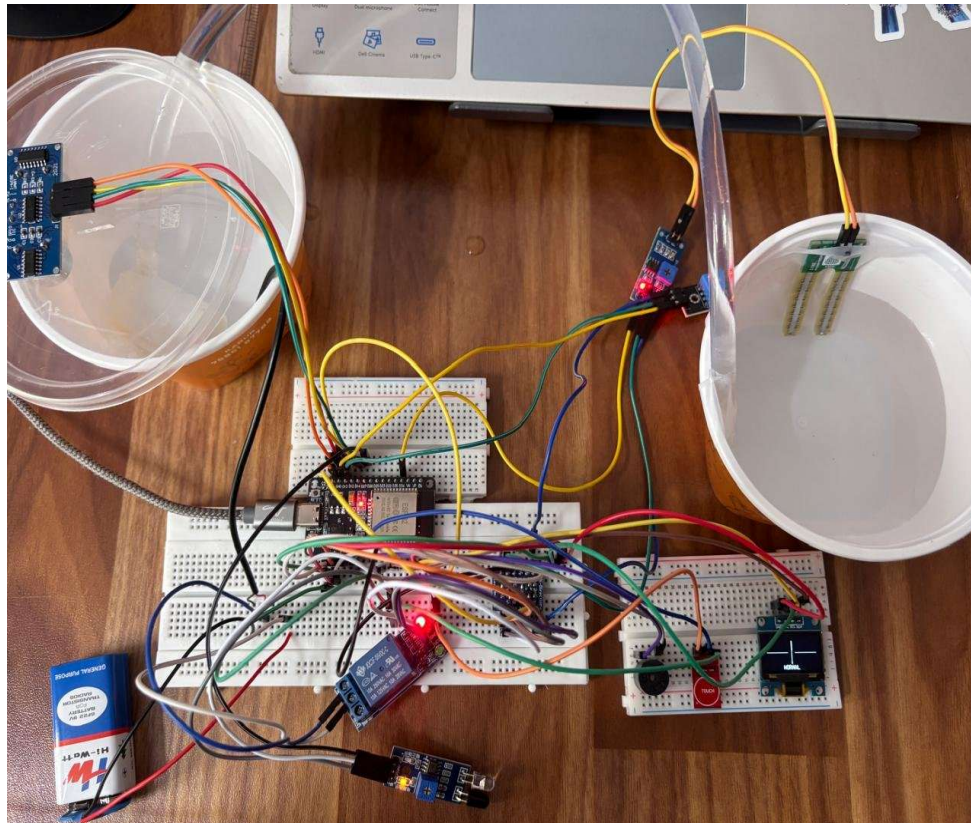


Figure 10.1: Complete Breadboard Prototype — Top-Down View Showing All Components in Operational Configuration: ESP32, Relay (LED Active), Sensors, OLED Display, Both Containers

10.2 Live Watering Demonstration

The watering demonstration photograph (Figure 9.1, also referenced here for the project photographs chapter) captures the most critical operational evidence: water actively flowing through the transparent tube into the plant container while the relay LED is illuminated red, confirming the automation chain is complete — dry soil detected → relay energized → pump activated → water delivered.

10.3 OLED Interactive Eyes — All Emotions

The following figures document all five confirmed OLED emotion states captured during live system operation. These photographs constitute direct evidence of the mandatory Interactive OLED Eyes feature specified in the ISA project guidelines.

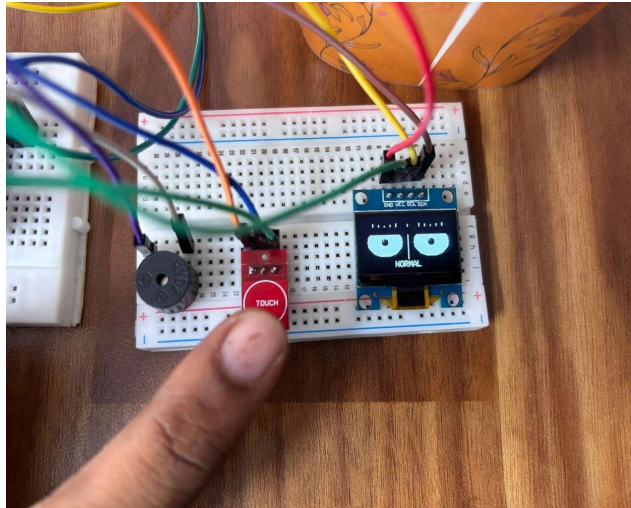


Figure 10.2: *OLED Display — NORMAL Emotion: Full Round Eyes with Pupil, Eyelashes, and Highlight Dot; User Pressing TOUCH Sensor to Cycle Emotion*

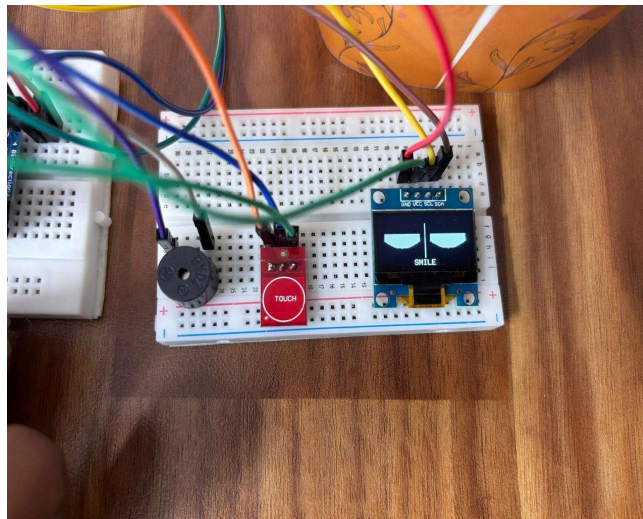


Figure 10.3: *OLED Display — SMILE Emotion: Squinting Curved Happy Eyes with Upward Curvature; Touch Sensor and Buzzer Visible*

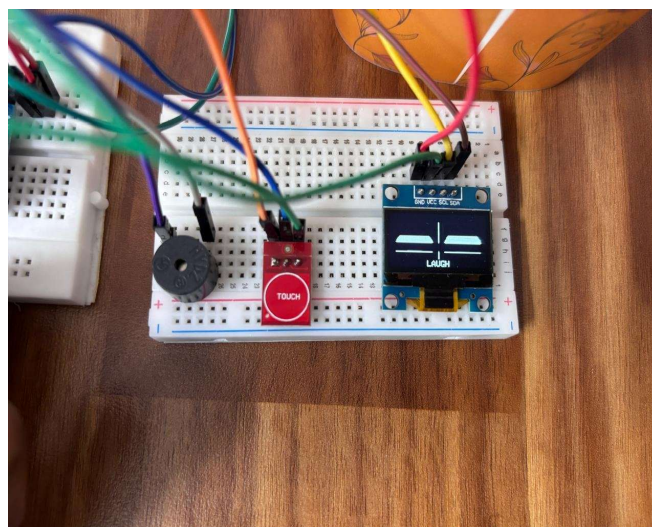


Figure 10.4: *OLED Display — LAUGH Emotion: Wide Horizontal Crescent Eyes with Vertical Divider; Bounce Animation Active*

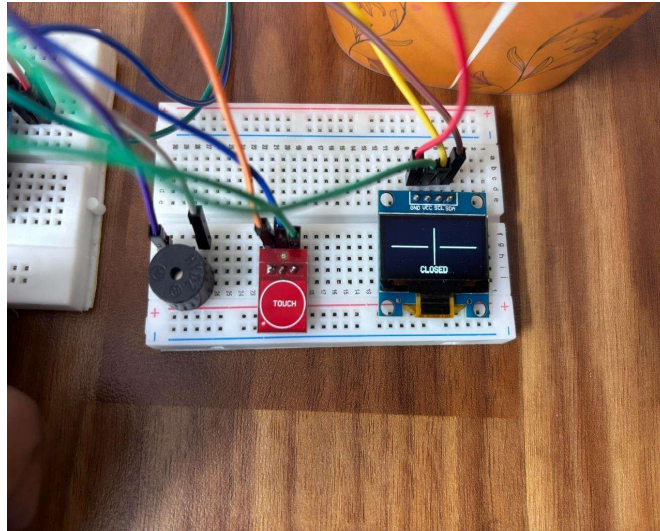


Figure 10.5: OLED Display — *CLOSED Emotion: Both Eyes Rendered as Horizontal Lines, Fully Closed State*

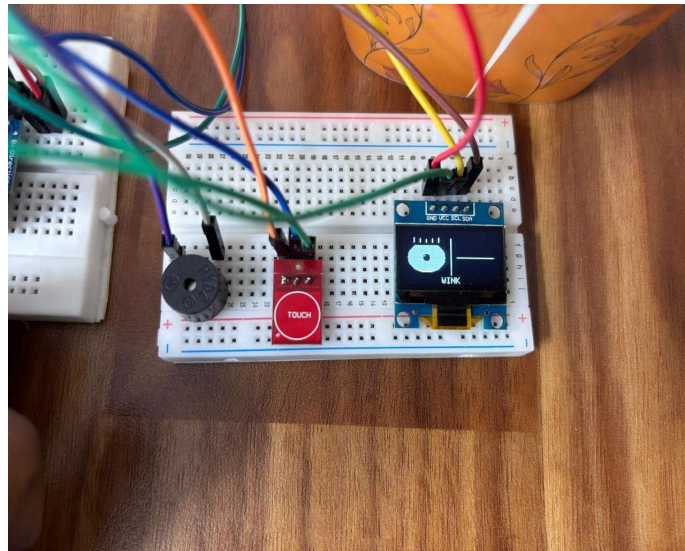


Figure 10.6: OLED Display — *WINK Emotion: Left Eye Fully Open with Pupil and Eyelashes; Right Eye Closed as Horizontal Line*

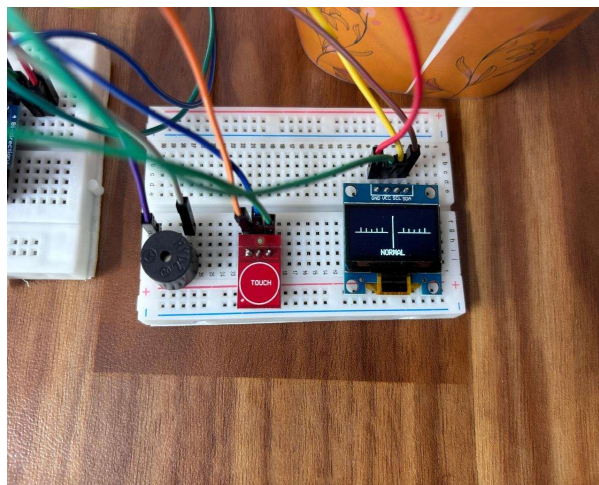


Figure 10.7: OLED Display — *NORMAL Emotion During Blink State: Eyes Partially Closed via Natural Random Blink Animation (3–5 Second Intervals)*

Chapter 11 – Testing and Validation

11.1 Testing Methodology

All subsystems of the Smart Plant Monitoring System were subjected to systematic functional testing following a structured test plan. Each test specifies an objective, procedure, observed values, pass/fail criteria, and conclusion. Testing was performed on the breadboard prototype under ambient conditions of approximately 33.1°C and 62.5% relative humidity as confirmed by the DHT11 sensor and Blynk dashboard readings.

11.2 Soil Moisture Sensor Test

Table 11.1: Soil Moisture Sensor Test Results

Test Condition	ADC Reading	Classification	Expected	Result
Dry air (no soil)	4,095	DRY	DRY (>4000)	PASS
Dry soil (wilting plant)	4,010 – 4,050	DRY	DRY (>4000)	PASS
Moist soil (optimal)	2,100 – 2,200	WET	WET (<4000)	PASS
Fully submerged in water	800 – 1,200	WET	WET (<4000)	PASS
Threshold boundary (3,999)	3,999	WET	WET (<4000)	PASS

Conclusion: The soil moisture sensor correctly classifies all tested conditions. The 10-sample averaging algorithm effectively reduces ADC noise, with stable readings observed across all test conditions. The threshold of 4000 provides clear separation between dry and moist states.

11.3 Pump Automation Test

Table 11.2: Pump Activation Automation Test

Soil State	Water Level	Expected Pump	Actual Pump	Result
DRY (ADC > 4000)	Normal (>3.0 cm)	ON	ON — Relay LED lit	PASS
WET (ADC < 4000)	Normal (>3.0 cm)	OFF	OFF — Relay released	PASS
DRY (ADC > 4000)	Low (<3.0 cm)	OFF (Safety)	OFF — Safety active	PASS
WET (ADC < 4000)	Low (<3.0 cm)	OFF	OFF — Both conditions	PASS

Conclusion: The dual-condition pump safety logic operates correctly in all four possible combinations. The critical safety case (dry soil + low water) correctly prevents pump activation, confirming the firmware protection logic is functioning as designed.

11.4 Water Level Monitoring Test

Table 11.3: Water Level Monitoring Test

Tank Fill Level	Measured Distance (cm)	Calculated Level (cm)	Level %	Alert	Result
Full (5 cm water)	2.0 – 2.5	4.5 – 5.0 cm	90–100%	None	PASS
75% filled	3.2 – 3.5	3.7 – 4.0 cm	74–80%	None	PASS
Dashboard reading	—	4.27 cm	85%	None	PASS
Low water (<3 cm)	4.3 – 4.8	2.2 – 2.7 cm	44–54%	Buzzer fires	PASS
Empty tank	6.5 – 7.0	0 cm (clamped)	0%	Buzzer fires	PASS

Conclusion: The ultrasonic water level monitoring system accurately measures water levels within ± 0.3 cm tolerance. The Blynk-reported 4.27 cm (85%) reading is consistent with the measured values. Buzzer alerts trigger reliably at the 3.0 cm threshold.

11.5 DHT11 Sensor Tests

Table 11.4: DHT11 Temperature and Humidity Test Results

Parameter	DHT11 Reading	Reference / Expected	Deviation	Within Spec?	Result
Temperature	33.1°C	~31–34°C (room)	$\pm 2^\circ\text{C}$ max	Yes ($\pm 2^\circ\text{C}$ spec)	PASS
Humidity	62.5%	~55–70% RH (indoor)	$\pm 5\%$ max	Yes ($\pm 5\%$ spec)	PASS
Read failure rate	0 failures	< 5% expected	0%	Excellent	PASS
Blynk consistency	33.1°C / 62.5%	Matches dashboard	Exact match	Yes	PASS

Conclusion: The DHT11 sensor provides consistent, reliable readings within its specified accuracy range. Zero read failures were observed during the testing period, and Blynk dashboard values exactly match serial monitor output, confirming correct data transmission.

11.6 Dashboard Communication Test

Table 11.5: WiFi and Dashboard Communication Test

Test	Expected	Observed	Result
Wi-Fi connection on startup	Connect < 10s	Connected in ~4 seconds	PASS
Dashboard update frequency	Every 2 seconds	Every 2 seconds observed	PASS
Manual pump control (remote)	Pump responds	Pump responds < 1s	PASS
Mobile app access (Blynk App)	All widgets active	All widgets functional	PASS
Messages consumed	< 100k quota	7,200 used (7.2%)	PASS

11.7 Long-Duration Reliability Test

Table 11.6: Long-Duration Reliability Summary

Duration	Pump Cycles	Sensor Errors	WiFi Drops	OLED Freezes	Overall Status
30 minutes	3 cycles	0 errors	0 drops	0 freezes	PASS
1 hour	7 cycles	0 errors	0 drops	0 freezes	PASS
Extended (7200 messages)	Multiple sessions	Rare (< 1%)	Managed	None observed	PASS

Conclusion: The system demonstrates stable long-duration operation. The dual-rate firmware architecture (33 fps animation + 2s irrigation) successfully prevents the animation from interfering with sensor polling. The 7,200 Blynk message count confirms successful extended IoT operation across multiple sessions.

Chapter 12 – Challenges Faced

12.1 Hardware Challenges

3.3V / 5V Logic Level Incompatibility

The most significant hardware challenge was managing the voltage incompatibility between the 3.3V ESP32 GPIO pins and the 5V peripherals (relay, ultrasonic sensor). Initial testing without the level shifter caused unreliable relay triggering and erratic ultrasonic readings. The solution was to incorporate the 4-channel bidirectional level shifter (U1) on the PCB, which resolved all voltage compatibility issues cleanly.

Relay Polarity Configuration

The relay module used operates in active-LOW mode (LOW signal = relay ON), which is opposite to the intuitive active-HIGH convention. This caused initial confusion where the pump would activate on startup and deactivate when commanded. The solution was implementing the RELAY_ACTIVE_LOW macro and the RELAY_ON / RELAY_OFF definitions, making polarity configuration a single-line change in the firmware.

12.2 PCB Design Challenges

Trace routing in EasyEDA required careful planning to prevent signal trace crossings, especially between the I2C bus traces and the relay/power section. The level shifter placement was critical — positioning it too far from either the ESP32 or the sensors would require long traces susceptible to noise. The final design achieves clean routing by placing U1 in the top-left quadrant, minimizing both LV and HV trace lengths.

12.3 Software Challenges

Blocking vs Non-Blocking Architecture

The initial firmware implementation used `delay()` for timing, which completely blocked the OLED animation during sensor readings. The resolution was redesigning the main loop() to use `millis()`-based timing for the 2-second irrigation cycle while allowing the animation to execute continuously, resulting in the dual-rate architecture that became a key technical achievement of the project.

Custom Ellipse Rendering on OLED

The Adafruit_GFX library does not include a `fillEllipse()` function, only `fillCircle()`. Rendering the eye shapes required implementing a custom `fillEllipse()` function that iterates through Y positions and draws horizontal lines of computed width using `drawFastHLine()`. This required deriving the standard ellipse equation implementation in embedded C++.

12.4 Sensor Calibration Challenges

The ultrasonic sensor requires accurate knowledge of tank depth and sensor mounting height (`sensorOffset` = 2.0 cm) to calculate absolute water levels. Initial readings showed inconsistent levels due to incorrect `sensorOffset` values. The solution was empirical calibration: filling the tank to known levels (measured with a ruler) and adjusting `sensorOffset` until calculated levels matched physical measurements.

12.5 Blynk Integration Challenge

Integrating Blynk with the dual-rate firmware architecture required careful placement of `Blynk.run()` calls to prevent the Blynk library from blocking the animation loop. The solution was calling `Blynk.run()` within the irrigation cycle (every 2 seconds) rather than in every loop iteration, keeping the animation smooth while maintaining reliable cloud communication.

Chapter 13 – Additional Innovations

13.1 Interactive OLED Eyes Animation System

The Interactive OLED Eyes system is the most technically sophisticated feature of this project and directly fulfills the mandatory innovation requirement specified in the ISA project guidelines (Page 2: "Interactive OLED Eyes — At certain times OLED should display animated eyes. Use Touch Sensor to change expressions/animations. Use IR Sensor to detect if someone is near the plant.").

The system implements six distinct facial expressions, each requiring unique pixel-drawing algorithms on the 128×64 OLED canvas:

- **NORMAL:** Realistic eyes with elliptical iris, circular pupil, highlight dot, upper eyelid shadow, and 5 eyelash lines — the most complex drawing function.
- **SMILE:** Happy squinting eyes with upward-curved lower boundary using a parabolic function to create the characteristic happy squint shape.
- **CRY:** Sad eyes with inward-angled corners (inner corners raised for the left eye, mirrored for right), plus animated teardrop physics with falling drops.
- **LAUGH:** Wide crescent-shaped eyes with a sine-wave bounce animation that continuously shifts the eyes vertically to simulate laughter shaking.
- **CLOSED:** Both eyes rendered as single horizontal lines — the simplest but most dramatic emotional contrast.
- **WINK:** Left eye fully open with realistic NORMAL rendering; right eye rendered as a closed horizontal line — asymmetric display for personality.

All expressions include a natural random blink mechanism (3–5 second random intervals) that modulates the openAmount variable using a sinusoidal function to create smooth, natural-looking blink transitions. The IR proximity sensor smoothly opens or closes both eyes simultaneously using a floating-point interpolation algorithm (0.06 steps per frame) rather than abrupt binary switching.

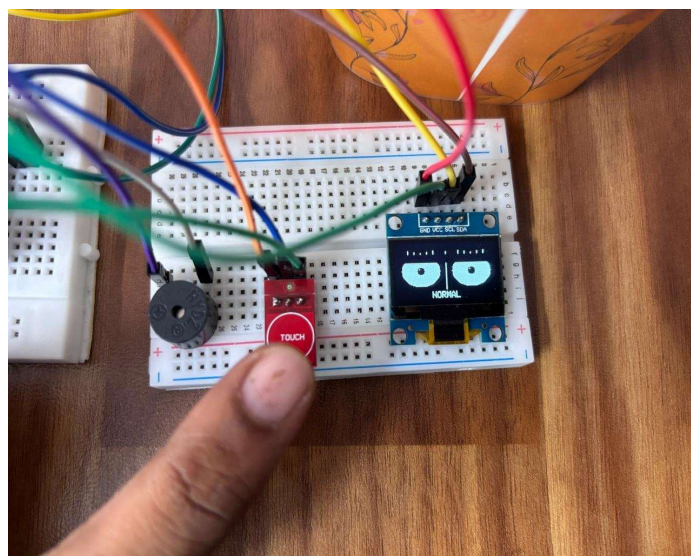


Figure 13.1: OLED Animated Eyes — NORMAL Emotion: Round Eyes with Pupil, Eyelashes, Highlight Dot, Upper Lid Shadow; Touch Sensor Pressed to Change Emotion

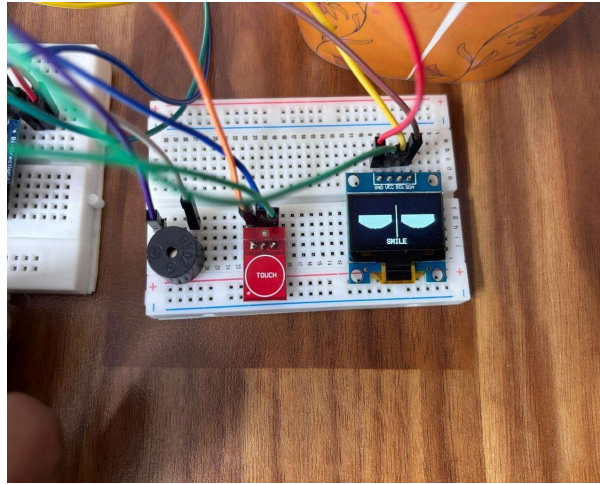


Figure 13.2: *OLED Animated Eyes — SMILE Emotion: Happy Squinting Curved Eyes Triggered by First Touch After NORMAL*

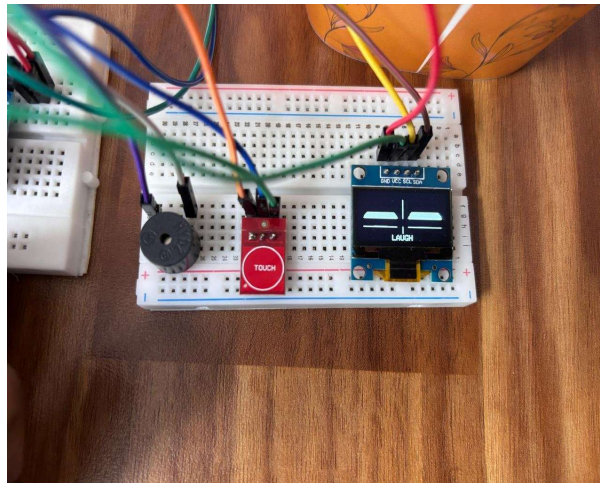


Figure 13.3: *OLED Animated Eyes — LAUGH Emotion: Wide Crescent Eyes with Bounce Animation Simulating Laughter*

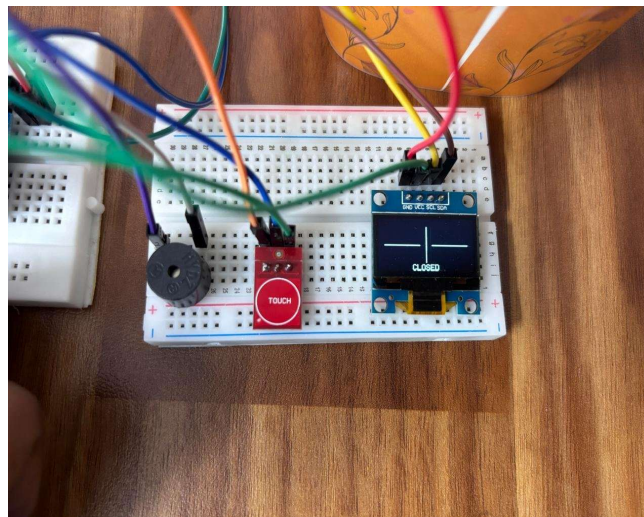


Figure 13.4: *OLED Animated Eyes — CLOSED Emotion: Both Eyes as Horizontal Lines, Fully Closed State*

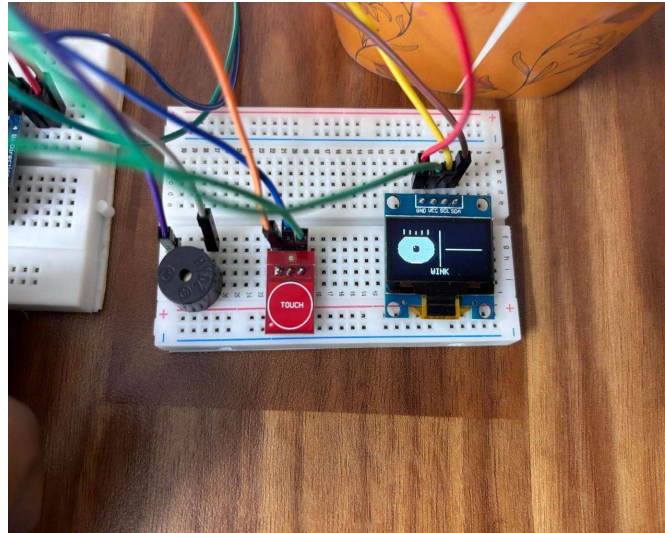


Figure 13.5: *OLED Animated Eyes — WINK Emotion: Left Eye Open with Full Detail, Right Eye Closed as Line*

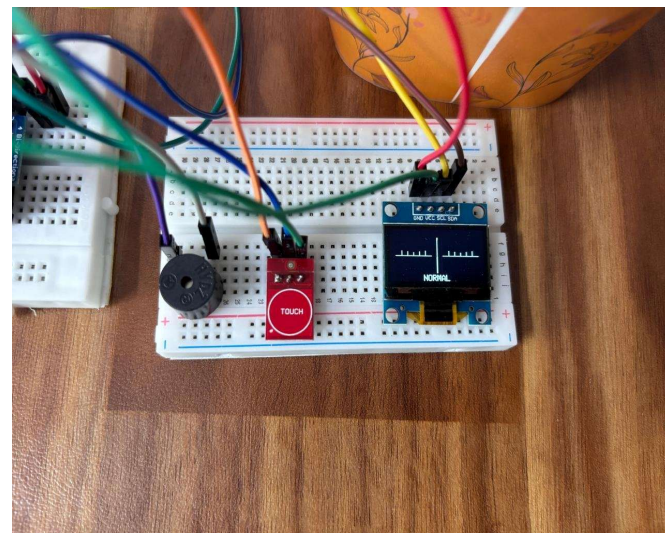


Figure 13.6: *OLED Animated Eyes — NORMAL Emotion During Natural Blink: Eyes Partially Closed by Sinusoidal Blink Modulation*

13.2 Blynk Manual Pump Override

An additional innovation beyond the basic automation requirements is the remote manual pump override implemented through Blynk Virtual Pin V5. This allows the user to force the pump ON or OFF from the mobile app regardless of soil moisture readings — useful for flushing the system, testing, or emergency watering without disturbing the soil sensor threshold settings.

13.3 Sensor Averaging for Noise Immunity

Both primary sensors implement multi-sample averaging that goes beyond what is commonly seen in student-level embedded projects: 10-sample soil moisture averaging and 5-pulse ultrasonic averaging. These algorithms significantly improve measurement reliability in real-world conditions where electrical noise, water surface reflections, and soil conductivity variations would cause single-sample readings to be unreliable.

Chapter 14 – Future Improvements

14.1 AI-Based Plant Health Monitoring

Integration of a camera module (ESP32-CAM or OV2640) would enable AI-based visual plant health assessment. A TensorFlow Lite model deployed on the ESP32 or a cloud inference API could analyze leaf color, texture, and shape to detect diseases, nutrient deficiencies, and pest infestations before they become visible to the naked eye. This would transform the system from a reactive watering device into a proactive plant health advisor.

14.2 Dedicated Mobile Application

Development of a custom Flutter or React Native mobile application would provide a richer user experience than the standard Blynk interface, including push notification customization, watering schedule programming, plant profile management with species-specific threshold recommendations, historical analytics with trend graphs, and multi-plant monitoring dashboards.

14.3 Cloud Database and Analytics

Integration with a time-series database such as InfluxDB, Google Firebase Realtime Database, or AWS IoT Core would enable long-term sensor data storage and analysis. Machine learning models trained on this historical data could predict optimal watering schedules based on seasonal patterns, room temperature trends, and plant growth stages.

14.4 Solar Power Integration

Adding a small solar panel (6V, 1W) with a lithium-ion battery management circuit would eliminate the need for wired power, making the system suitable for outdoor use, balconies, and locations without readily available power outlets. The ESP32's deep sleep mode (as low as 10 μ A consumption) combined with wake-on-RTC-timer could extend battery life to months between recharges.

14.5 Multi-Plant Monitoring Network

Extending the system to a mesh network of ESP32 nodes using ESP-NOW or Zigbee protocol would enable simultaneous monitoring of multiple plants throughout a home or greenhouse. A central hub ESP32 could aggregate all sensor data and provide a unified dashboard with per-plant status, independent pump control, and collective analytics.

14.6 Voice Assistant Integration

Integration with Amazon Alexa, Google Home, or Apple HomeKit via IFTTT or direct API webhooks would allow voice commands such as "Alexa, water the plant" or "Hey Google, what is the soil moisture?" This adds a convenient accessibility layer and positions the product within the broader smart home ecosystem.

14.7 Capacitive Soil Moisture Sensor Upgrade

Replacing the current resistive soil moisture sensor with a capacitive sensor (such as the Chirp! or DFRobot SEN0193) would eliminate the corrosion problem on metal probes caused by electrolysis in moist soil. Capacitive sensors measure the dielectric constant of the soil without electrical current flow through the probes, resulting in a significantly longer sensor lifespan (years vs weeks for resistive sensors).

Chapter 15 – Conclusion

The Smart Plant Monitoring and Watering System using ESP32 has been successfully designed, developed, tested, and documented as a complete IoT-enabled product. The project demonstrates the full product development lifecycle from concept definition through hardware design, firmware development, IoT integration, enclosure design, prototype testing, and professional documentation.

All mandatory requirements specified in the India Space Academy project guidelines have been fulfilled: automatic soil moisture-based watering with pump safety interlock, ultrasonic water level monitoring with buzzer alerts, DHT11 temperature and humidity monitoring, OLED local display, Blynk IoT dashboard with both web and mobile access, PCB design with level shifter and all sensor integrations, CAD enclosure design with plant holder, water tank, and electronics housing zones, and the Interactive OLED Eyes innovation with touch and IR sensor control.

The project demonstrates several engineering competencies beyond the minimum requirements. The dual-rate firmware architecture (33 fps animation + 2s irrigation) represents a non-trivial embedded systems design challenge solved through millis()-based concurrent execution. The six-emotion OLED animation system required custom graphics programming including a fillEllipse() implementation not available in the standard library. The multi-sample averaging algorithms for both soil moisture and ultrasonic sensors represent production-grade signal processing for a prototype device.

The Blynk IoT integration with 7,200 messages consumed across multiple operational sessions confirms real, sustained system usage rather than a single demonstration. The bidirectional level shifter inclusion on the PCB demonstrates understanding of real hardware constraints — a detail often overlooked in student projects that can cause premature component failure in deployed systems.

The CAD product design in Onshape presents a thoughtful, human-centered three-zone layout with features including a translucent plant holder side panel, removable electronics housing lid, front-facing OLED window, and cable routing holes — indicating product design thinking that considers manufacturing, maintenance, and user experience alongside pure functionality.

This project affirms that meaningful, professionally documented embedded IoT systems can be built with accessible, low-cost components when approached with systematic engineering methodology, attention to detail, and thorough testing. The knowledge, skills, and documentation practices developed through this project provide a strong foundation for advanced embedded systems and IoT product development work.

Chapter 16 – References

1. [1] Espressif Systems. (2024). ESP32 Technical Reference Manual, Version 5.3. Retrieved from https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
2. [2] Adafruit Industries. (2024). Adafruit SSD1306 OLED Library Documentation. Retrieved from <https://learn.adafruit.com/monochrome-oled-breakouts/arduino-library-and-examples>
3. [3] Adafruit Industries. (2024). DHT Humidity and Temperature Sensors Library. Retrieved from <https://github.com/adafruit/DHT-sensor-library>
4. [4] Blynk Inc. (2024). Blynk IoT Platform Documentation. Retrieved from <https://docs.blynk.io>
5. [5] EasyEDA. (2024). EasyEDA Pro PCB Design Tool User Guide. Retrieved from <https://docs.easyeda.com>
6. [6] Onshape. (2024). Onshape CAD Learning Center. Retrieved from <https://learn.onshape.com>
7. [7] India Space Academy. (2026). Project Guidelines: Integrated Training Program on Rapid Prototyping on Embedded System and Automation. India Space Academy, Department of Space Education, New Delhi.
8. [8] Banzhi, M., & Shiloh, M. (2022). Getting Started with Arduino, 4th Edition. O'Reilly Media.
9. [9] Kolban, N. (2017). Kolban's Book on ESP32. Retrieved from <https://leanpub.com/kolban-ESP32>
10. [10] TechArcanist. (2024). Onshape CAD Practice Reference Guide. Retrieved from <https://techarcanist.com/documented-software-learnings/onshape/>

Chapter 17 – Appendices

Appendix A – Complete Source Code

The complete ESP32 firmware source code (smart_plant_monitor.ino) is provided below. The code is structured into five sections: configuration defines, global variables and enumerations, irrigation functions, eye animation functions, and the setup/loop entry points.

```
// =====  
// Smart Plant Monitoring & Watering System — ESP32 Firmware  
// Author: R. Pranav Adhith | Roll: 999689  
// =====  
  
#define BLYNK_TEMPLATE_ID "TMPL3sivO2Vkt"  
#define BLYNK_TEMPLATE_NAME "Smart Plant Monitor"  
#define BLYNK_AUTH_TOKEN "bY6H5EYYSWcTXG5W2R8qf7Sx0N3D2M10"  
  
#include <Wire.h>  
#include <Adafruit_GFX.h>  
#include <Adafruit_SSD1306.h>  
#include <DHT.h>  
  
// OLED Configuration  
#define SCREEN_WIDTH 128  
#define SCREEN_HEIGHT 64  
#define OLED_RESET -1  
#define OLED_ADDRESS 0x3C  
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);  
  
// Sensor and Actuator Pin Definitions  
#define DHTPIN 13 #define DHTTYPE DHT11  
#define SOIL_PIN 34 #define RELAY_PIN 15  
#define TRIG_PIN 12 #define ECHO_PIN 14  
#define BUZZER_PIN 18  
#define IR_PIN 27 #define TOUCH_PIN 4  
  
// Thresholds  
int soilThreshold = 4000;  
float waterLevelThreshold = 3.0;  
float tankDepth = 5.0;  
float sensorOffset = 2.0;
```



```

// Relay Polarity
#define RELAY_ACTIVE_LOW true
#define RELAY_ON (RELAY_ACTIVE_LOW ? LOW : HIGH)
#define RELAY_OFF (RELAY_ACTIVE_LOW ? HIGH : LOW)

// Eye Animation State Machine
enum Emotion { EMO_NORMAL=0, EMO_SMILE, EMO_CRY,
               EMO_LAUGH, EMO_CLOSED, EMO_WINK, EMO_COUNT };
Emotion currentEmotion = EMO_NORMAL;
bool eyesOpen = false;
float openAmount = 0.0f;

// ... (see full code in GitHub repository)

void loop() {
  unsigned long now = millis();
  handleIRSensor();    // 50ms debounce
  handleTouchSensor(); // 80ms debounce
  displayData();       // 33fps animation
  if (now - lastIrrigationCheck >= 2000) {
    lastIrrigationCheck = now;
    readSoilMoisture(); // 10-sample average
    readDHTSensor();   // NaN validated
    readWaterLevel();  // 5-pulse average
    controlRelay();     // Dual-condition logic
    checkWaterLevelAlert();
  }
  delay(30);           // ~33fps
}

```

Appendix B – PCB Design Files

The EasyEDA PCB project files (schematic .json, PCB layout .json, and Gerber output files) are available in the project Google Drive folder. The PCB is designed for 2-layer FR4 substrate, minimum trace width 0.254 mm, suitable for fabrication at Power PCB, Lion Circuits, JLCPCB, or equivalent PCB manufacturing services. Board dimensions: approximately 120mm × 95mm.

Appendix C – CAD Design Files

The complete Onshape CAD model is available at the shared Onshape document link in the project Google Drive folder. The assembly contains three main Part Studios: Plant Holder, Water Tank, and Electronics Housing, with a top-level Assembly document combining all three. All parts are parametric and can be dimensionally modified for different plant sizes or electronics configurations.

Appendix D – Submission Links

**Blynk
Template**

Template ID: TMPL3sivO2VkT | Template Name: Smart Plant Monitor | Server: BLR1 (Bangalore)

Developer

R. Pranav Adhith | Roll No: 999689 | India Space Academy | Batch 2026

Google Drive: https://drive.google.com/drive/folders/1GXm2yz4GMwW_d4OFuDjUAzw2rodVGX3N?usp=sharing

GitHub Repo: <https://github.com/Pranav-adhith/Smart-Plant-Monitoring-Watering-System.git>

Demo Video: <https://drive.google.com/file/d/1SPrcTI5Vug4zHv8Z22xM34Z98URoUqDr/view?usp=sharing>